AG3 - Actividad Guiada 3

Nombre: Ventura Esteban Guerrero
Link: https://colab.research.google.com/drive/1qqgOd8ntmcRdZpPHuhaLfc5_36YJWMBv?usp=sharing
Github: https://github.com/venturaesteban/03MIAR-Algoritmos-de-Optimizaci-n

Carga de librerias

Haz doble clic (o pulsa Intro) para editar

```
!pip install requests
!pip install tabulate==0.8.7
!pip install networkx==2.8.8
!pip install deprecated==1.2.9
!pip install tsplib95 --no-deps
```

```
Requirement already satisfied: requests in /usr/local/lib/python3.12/dist-packages (2.32.4)
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests) (3.4.4)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests) (3.11)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests) (2.5.0)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests) (2026.2.25)
Requirement already satisfied: tabulate==0.8.7 in /usr/local/lib/python3.12/dist-packages (0.8.7)
Requirement already satisfied: networkx==2.8.8 in /usr/local/lib/python3.12/dist-packages (2.8.8)
Requirement already satisfied: deprecated==1.2.9 in /usr/local/lib/python3.12/dist-packages (1.2.9)
Requirement already satisfied: wrapt<2,>=1.10 in /usr/local/lib/python3.12/dist-packages (from deprecated==1.2.9) (1.17.3)
Requirement already satisfied: tsplib95 in /usr/local/lib/python3.12/dist-packages (0.7.1)
```

```
!pip install deprecated
```

```
Requirement already satisfied: deprecated in /usr/local/lib/python3.12/dist-packages (1.2.9)
Requirement already satisfied: wrapt<2,>=1.10 in /usr/local/lib/python3.12/dist-packages (from deprecated) (1.17.3)
```

Carga de los datos del problema

```
import urllib.request #Hacer llamadas http a paginas de la red
import tsplib95       #Modulo para las instancias del problema del TSP
import math           #Modulo de funciones matematicas. Se usa para exp
import random         #Para generar valores aleatorios


#Descargamos el fichero de datos(Matriz de distancias)
file = "swiss42.tsp" ;
#urllib.request.urlretrieve("https://raw.githubusercontent.com/mastqe/tsplib/refs/heads/master/swiss42.tsp", file + '.gz')
urllib.request.urlretrieve("https://raw.githubusercontent.com/mastqe/tsplib/refs/heads/master/swiss42.tsp", file  )
#!gzip -d swiss42.tsp.gz     #Descomprimir el fichero de datos

#Coordendas 51-city problem (Christofides/Eilon)
#file = "eil51.tsp" ; urllib.request.urlretrieve("http://comopt.ifi.uni-heidelberg.de/software/TSPLIB95/tsp/eil51.tsp.gz",

#Coordenadas - 48 capitals of the US (Padberg/Rinaldi)
#file = "att48.tsp" ; urllib.request.urlretrieve("http://comopt.ifi.uni-heidelberg.de/software/TSPLIB95/tsp/att48.tsp.gz",


#http://comopt.ifi.uni-heidelberg.de/software/TSPLIB95
#Documentacion :
  # http://comopt.ifi.uni-heidelberg.de/software/TSPLIB95/tsp95.pdf
  # https://tsplib95.readthedocs.io/en/stable/pages/usage.html
  # https://tsplib95.readthedocs.io/en/v0.6.1/modules.html
  # https://pypi.org/project/tsplib95/
```

```
('swiss42.tsp', <http.client.HTTPMessage at 0x7c451c801340>)
```

```
#Carga de datos y generación de objeto problem
##############################################################################
problem = tsplib95.load(file)

#Nodos
```

```
Nodos = list(problem.get_nodes())

#Aristas
Aristas = list(problem.get_edges())
```

```
Nodos
```

```
Aristas
# https://github.com/mastqe/tsplib/blob/master/swiss42.tsp
```

```
NOMBRE: swiss42
TIPO: TSP
COMENTARIO: 42 Staedte Schweiz (Fricker)
DIMENSION: 42
EDGE_WEIGHT_TYPE: EXPLICIT
EDGE_WEIGHT_FORMAT: FULL_MATRIX
EDGE_WEIGHT_SECTION
0 15 30 23 32 55 33 37 92 114 92 110 96 90 74 76 82 72 78 82 159 122 131 206 112 57 28 43 70
  15 0 34 23 27 40 19 32 93 117 88 100 87 75 63 67 71 69 62 63 96 164 132 131 212 106 44 33 5
  30 34 0 11 18 57 36 65 62 84 64 89 76 93 95 100 104 98 57 88 99 130 100 101 179 86 51 4 18
  23 23 11 0 11 48 26 54 70 94 69 75 75 84 84 89 92 89 54 78 99 141 111 109 89 89 11 11 11 54
  32 27 18 11 0 40 20 58 67 92 61 78 65 76 83 89 91 95 43 72 110 141 116 105 190 81 34 19 35
  55 40 57 48 40 0 23 55 96 123 78 75 36 36 66 66 63 95 34 34 137 174 156 129 224 90 15 59 75
  33 19 36 26 20 23 0 45 85 111 75 82 69 60 63 70 71 85 44 52 115 161 136 122 210 91 25 37 54
  37 32 65 54 58 55 45 0 124 149 118 126 113 80 42 42 40 40 87 87 94 158 158 163 242 135 65 6
  92 93 62 70 67 96 85 124 0 28 29 68 63 122 148 155 156 159 67 129 148 78 80 39 129 46 82 65
 114 117 84 94 92 123 111 149 28 0 54 91 88 150 174 181 182 181 95 157 159 50 65 27 102 65 11
  92 88 64 69 61 78 75 118 29 54 0 39 34 99 134 142 141 157 44 110 161 103 109 52 154 22 63 6
 110 100 89 89 78 75 82 126 68 91 39 0 14 80 129 139 135 167 39 98 187 136 148 81 186 28 61 9
  96 87 76 75 65 62 69 113 63 88 34 14 0 72 117 128 124 153 26 88 174 136 142 82 187 32 48 79
  90 75 93 84 76 36 60 80 122 150 99 80 72 0 59 71 63 116 56 25 170 201 189 151 252 104 44 95
  74 63 95 84 83 56 63 42 148 174 134 129 117 59 0 11 8 63 93 35 135 223 195 184 273 146 71 9
```

```
#Probamos algunas funciones del objeto problem

#Distancia entre nodos
problem.get_weight(5, 20)

#Todas las funciones
#Documentación: https://tsplib95.readthedocs.io/en/v0.6.1/modules.html

#dir(problem)
```

```
137
```

Funcionas basicas

```
#Funcionas basicas
##############################################################################

#Se genera una solucion aleatoria con comienzo en en el nodo 0
def crear_solucion(Nodos):
  solucion = [Nodos[0]]
  for n in Nodos[1:]:
    solucion = solucion + [random.choice(list(set(Nodos) - set({Nodos[0]}) - set(solucion)))]
  return solucion

#Devuelve la distancia entre dos nodos
def distancia(a,b, problem):
  return problem.get_weight(a,b)

#Devuelve la distancia total de una trayectoria/solucion
def distancia_total(solucion, problem):
  distancia_total = 0
  for i in range(len(solucion)-1):
    distancia_total += distancia(solucion[i] ,solucion[i+1] ,  problem)
  return distancia_total + distancia(solucion[len(solucion)-1] ,solucion[0], problem)

sol_temporal = crear_solucion(Nodos)


print(sol_temporal   )
print(distancia_total(sol_temporal, problem))
```

```
[0, 39, 23, 2, 10, 9, 40, 8, 37, 26, 15, 24, 31, 21, 38, 7, 33, 6, 1, 14, 27, 34, 30, 12, 19, 4, 22, 35, 20, 32, 13, 16, 3,
4394
```

BUSQUEDA ALEATORIA

```
################################################################################
# BUSQUEDA ALEATORIA
################################################################################

def busqueda_aleatoria(problem, N):
  #N es el numero de iteraciones
  Nodos = list(problem.get_nodes())

  mejor_solucion = []
  #mejor_distancia = 10e100                        #Inicializamos con un valor alto
  mejor_distancia = float('inf')                   #Inicializamos con un valor alto

  for i in range(N):                               #Criterio de parada: repetir N veces pero podemos incluir otros
    solucion = crear_solucion(Nodos)               #Genera una solucion aleatoria
    distancia = distancia_total(solucion, problem)  #Calcula el valor objetivo(distancia total)

    if distancia < mejor_distancia:                #Compara con la mejor obtenida hasta ahora
      mejor_solucion = solucion
      mejor_distancia = distancia


  print("Mejor solución:" , mejor_solucion)
  print("Distancia     :" , mejor_distancia)
  return mejor_solucion


#Busqueda aleatoria con 5000 iteraciones
solucion = busqueda_aleatoria(problem, 50000)
```

```
Mejor solución: [0, 17, 16, 19, 29, 4, 15, 36, 13, 12, 25, 41, 30, 26, 6, 18, 33, 2, 27, 3, 37, 10, 35, 34, 32, 24, 40, 9, 2
Distancia     : 3366
```

BUSQUEDA LOCAL

```
################################################################################
# BUSQUEDA LOCAL(1 paso)
################################################################################
def genera_vecina(solucion):
  #Generador de soluciones vecinas: 2-opt (intercambiar 2 nodos) Si hay N nodos se generan (N-1)x(N-2)/2 soluciones
  #Se puede modificar para aplicar otros generadores distintos que 2-opt
  #print(solucion)
  mejor_solucion = []
  mejor_distancia = 10e100
  for i in range(1,len(solucion)-1):          #Recorremos todos los nodos en bucle doble para evaluar todos los intercambio
    for j in range(i+1, len(solucion)):

      #Se genera una nueva solución intercambiando los dos nodos i,j:
      #  (usamos el operador + que para listas en python las concatena) : ej.: [1,2] + [3] = [1,2,3]
      vecina = solucion[:i] + [solucion[j]] + solucion[i+1:j] + [solucion[i]] + solucion[j+1:]

      #Se evalua la nueva solución ...
      distancia_vecina = distancia_total(vecina, problem)

      #... para guardarla si mejora las anteriores
      if distancia_vecina <= mejor_distancia:
        mejor_distancia = distancia_vecina
        mejor_solucion = vecina
  return mejor_solucion


#solucion = [1, 47, 13, 41, 40, 19, 42, 44, 37, 5, 22, 28, 3, 2, 29, 21, 50, 34, 30, 9, 16, 11, 38, 49, 10, 39, 33, 45, 15,
print("Distancia Solucion Incial:" , distancia_total(solucion, problem))


nueva_solucion = genera_vecina(solucion)
print("Distancia Mejor Solucion Local:", distancia_total(nueva_solucion, problem))
```

```
Distancia Solucion Incial: 3366
Distancia Mejor Solucion Local: 3052
```

```
#Busqueda Local(iteraciones):
#  - Sobre el operador de vecindad 2-opt(funcion genera_vecina)
#  - Sin criterio de parada, se para cuando no es posible mejorar.
def busqueda_local(solucion, problem):
```

```
  mejor_solucion = []

  #Generar una solucion inicial de referencia(aleatoria)
  #solucion_referencia = crear_solucion(Nodos)
  solucion_referencia = solucion
  mejor_distancia = distancia_total(solucion_referencia, problem)

  iteracion=0             #Un contador para saber las iteraciones que hacemos
  while(1):
    iteracion +=1         #Incrementamos el contador
    #print('#',iteracion)

    #Obtenemos la mejor vecina ...
    vecina = genera_vecina(solucion_referencia)

    #... y la evaluamos para ver si mejoramos respecto a lo encontrado hasta el momento
    distancia_vecina = distancia_total(vecina, problem)

    #Si no mejoramos hay que terminar. Hemos llegado a un minimo local(según nuestro operador de vencindad 2-opt)
    if distancia_vecina < mejor_distancia:
      #mejor_solucion = copy.deepcopy(vecina)   #Con copia profunda. Las copias en python son por referencia
      mejor_solucion = vecina                   #Guarda la mejor solución encontrada
      mejor_distancia = distancia_vecina

    else:
      print("En la iteracion ", iteracion, ", la mejor solución encontrada es:" , mejor_solucion)
      print("Distancia     :" , mejor_distancia)
      return mejor_solucion

    solucion_referencia = vecina


sol = busqueda_local(nueva_solucion, problem )
```

```
En la iteracion  27 , la mejor solución encontrada es: [0, 14, 16, 19, 13, 5, 6, 26, 18, 12, 11, 25, 10, 8, 28, 27, 3, 4, 1,
Distancia     : 1797
```

SIMULATED ANNEALING

```
##############################################################################
# SIMULATED ANNEALING
##############################################################################

#Generador de 1 solucion vecina 2-opt 100% aleatoria (intercambiar 2 nodos)
#Mejorable eligiendo otra forma de elegir una vecina.
def genera_vecina_aleatorio(solucion):

  #Se eligen dos nodos aleatoriamente
  i,j = sorted(random.sample( range(1,len(solucion)) , 2))

  #Devuelve una nueva solución pero intercambiando los dos nodos elegidos al azar
  return solucion[:i] + [solucion[j]] + solucion[i+1:j] + [solucion[i]] + solucion[j+1:]


#Funcion de probabilidad para aceptar peores soluciones
def probabilidad(T,d):
  if random.random() <  math.exp( -1*d / T)  :
    return True
  else:
    return False

#Funcion de descenso de temperatura
def bajar_temperatura(T):
  return T*0.99
```

```
def recocido_simulado(problem, TEMPERATURA ):
  #problem = datos del problema
  #T = Temperatura

  solucion_referencia = crear_solucion(Nodos)
  distancia_referencia = distancia_total(solucion_referencia, problem)

  mejor_solucion = []             #x* del seudocodigo
  mejor_distancia = 10e100        #F* del seudocodigo


  N=0
  while TEMPERATURA > .0001:
    N+=1
```

```
        #Genera una solución vecina
        vecina =genera_vecina_aleatorio(solucion_referencia)

        #Calcula su valor(distancia)
        distancia_vecina = distancia_total(vecina, problem)

        #Si es la mejor solución de todas se guarda(siempre!!!)
        if distancia_vecina < mejor_distancia:
            mejor_solucion = vecina
            mejor_distancia = distancia_vecina

        #Si la nueva vecina es mejor se cambia
        #Si es peor se cambia según una probabilidad que depende de T y delta(distancia_referencia - distancia_vecina)
        if distancia_vecina < distancia_referencia or probabilidad(TEMPERATURA, abs(distancia_referencia - distancia_vecina) )
          #solucion_referencia = copy.deepcopy(vecina)
          solucion_referencia = vecina
          distancia_referencia = distancia_vecina

        #Bajamos la temperatura
        TEMPERATURA = bajar_temperatura(TEMPERATURA)

      print("La mejor solución encontrada es " , end="")
      print(mejor_solucion)
      print("con una distancia total de " , end="")
      print(mejor_distancia)
      return mejor_solucion

    sol  = recocido_simulado(problem, 100000000)
```

```
La mejor solución encontrada es [0, 1, 4, 5, 26, 12, 13, 19, 31, 36, 35, 33, 38, 24, 40, 21, 9, 28, 27, 2, 3, 18, 11, 25, 10
con una distancia total de 1925
```

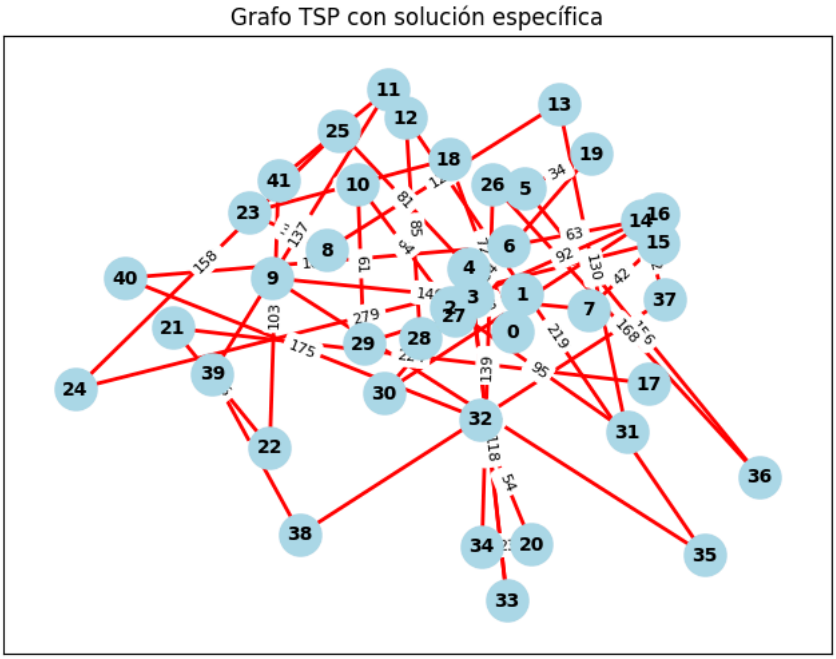
Representación en un grafo a partir de la matriz de distancias(Optimización de posiciones usando escalado multidimensional (MDS)

Multidimensional scaling problem(MDS): https://en.wikipedia.org/wiki/Multidimensional_scaling

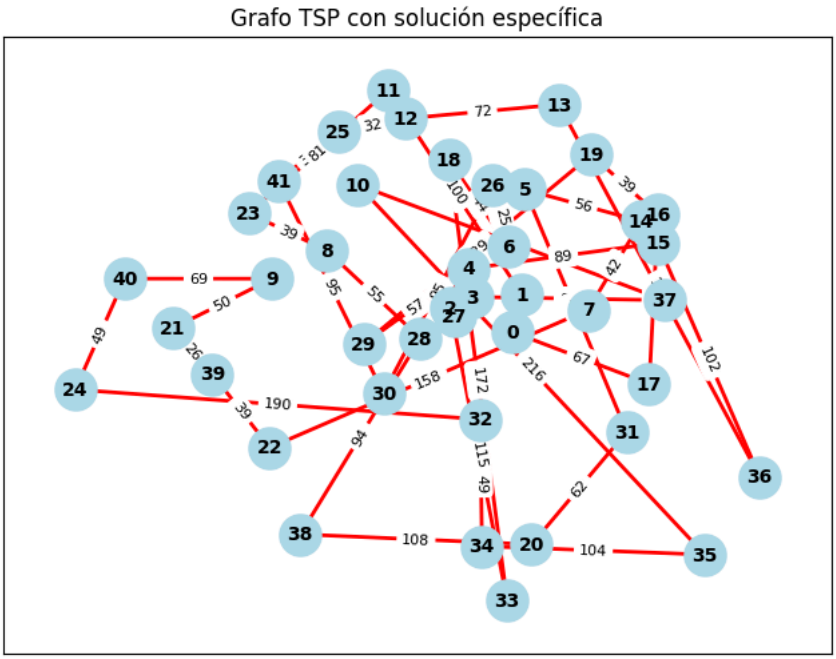
```
import numpy as np
import networkx as nx
import matplotlib.pyplot as plt
from sklearn.manifold import MDS  # Multidimensional Scaling o Escalado Multidimensional

def plot_tsp_solution(distance_matrix, tsp_solution):
    """
    Dibuja el grafo de un TSP con las posiciones calculadas mediante MDS y muestra
    solo las aristas correspondientes a la solución del TSP.

    :param distance_matrix: np.ndarray, matriz de distancias entre nodos
    :param tsp_solution: list, lista de nodos en el orden de la solución del TSP
    """
    # Crear el grafo completo
    G = nx.Graph()
    num_nodes = len(distance_matrix)
    for i in range(num_nodes):
        for j in range(i + 1, num_nodes):
            G.add_edge(i, j, weight=distance_matrix[i][j])

    # Usar MDS para calcular posiciones de los nodos
    mds = MDS(n_components=2, dissimilarity="precomputed", random_state=42)
    positions = mds.fit_transform(distance_matrix)

    # Convertir las posiciones en un diccionario para networkx
    pos = {i: positions[i] for i in range(num_nodes)}

    # Crear un subgrafo con las aristas del camino TSP
    TSP_G = nx.Graph()
    for i in range(len(tsp_solution) - 1):
        u = tsp_solution[i]
        v = tsp_solution[i + 1]
        TSP_G.add_edge(u, v, weight=distance_matrix[u][v])

    # Dibujar el grafo
    plt.figure(figsize=(8, 6))

    # Dibujar nodos
    nx.draw_networkx_nodes(G, pos, node_color='lightblue', node_size=500)
```

```
    # Dibujar las aristas del camino TSP
    nx.draw_networkx_edges(TSP_G, pos, edge_color='red', width=2)

    # Añadir etiquetas a los nodos y pesos de las aristas
    nx.draw_networkx_labels(G, pos, font_size=10, font_weight='bold')
    edge_labels = nx.get_edge_attributes(TSP_G, 'weight')
    nx.draw_networkx_edge_labels(TSP_G, pos, edge_labels=edge_labels, font_size=8)

    plt.title("Grafo TSP con solución específica")
    plt.show()
```

```
plot_tsp_solution(problem.edge_weights, crear_solucion(Nodos))
```




```
plot_tsp_solution(problem.edge_weights, solucion)
```



Búsqueda Local con Entornos Variables

¿Se puede mejorar con otros operadores de vecindad variables?

La búsqueda local que implementamos antes usa solo el operador 2-opt (intercambio de 2 nodos). La idea de la **Búsqueda por Entornos Variables (VNS)** es usar varios operadores de vecindad distintos con el fin de ir probando con distintos nodos para obtener una solución diferente si con el inicial ya no se puede mejorar.

Tal y como mencionó el profesor en la clase, hemos implementado el concepto **n-opt** variando n. Usamos el operador **or-opt**, que coge un segmento de n nodos consecutivos y lo reubica en otra posición del recorrido, empezando con segmentos grandes (n=5) que hacen cambios más bruscos e ir bajando hasta n=2 que son cambios más finos. Cada vez que se encuentra una mejora, volvemos al segmento más grande (n=5).

```
def genera_vecina_or_opt(solucion, n):
  """
  Vamos a usar el operador or-opt, que coge un segmento de n nodos consecutivos
  y lo reubica en la mejor posición posible.
  Usaremos n=5 para cambios grandes y bajaremos a n=2 para cambios finos
  """
  mejor_solucion = []
  mejor_distancia = 10e100
  num_nodos = len(solucion)

  for i in range(1, num_nodos - n + 1):            # Posición del segmento a extraer (trato el nodo 0 como fijo)
    segmento = solucion[i:i+n]
    resto = solucion[:i] + solucion[i+n:]  # El recorrido sin ese segmento (que se reinsertará después)

    for j in range(1, len(resto) + 1):      # Posiciones donde reinsertar el segmento (entre nodos del resto)
      vecina = resto[:j] + segmento + resto[j:]

      distancia_vecina = distancia_total(vecina, problem)
      if distancia_vecina <= mejor_distancia:
        mejor_distancia = distancia_vecina
        mejor_solucion = vecina

  return mejor_solucion
```

```
def busqueda_local_VNS(solucion_inicial, problem):
  """
  Usamos varios operadores n-opt (de n=5 a n=2) + el original.
    Si un operador no mejora, pasamos al siguiente.
    Si alguno mejora, volvemos al primero (n=5) y repetimos hasta que ninguno mejore.
  """
  solucion = solucion_inicial
  mejor_distancia = distancia_total(solucion, problem)

  # Operadores ordenados de mayor a menor
  nombres = ["or-opt(5)", "or-opt(4)", "or-opt(3)", "or-opt(2)", "2-opt"]

  n = 0  # Empezamos con el primer operador (or-opt n=5)
  iteracion = 0

  while n < 5:
    iteracion += 1

    # Elegimos el operador según n
    if n < 4:
      vecina = genera_vecina_or_opt(solucion, 5 - n)
    else:
      vecina = genera_vecina(solucion)        # El último operador es el 2-opt original

    distancia_vecina = distancia_total(vecina, problem)

    if distancia_vecina < mejor_distancia:
      print(f"  Iteracion {iteracion}: Mejora con {nombres[n]} -> distancia = {distancia_vecina}")
      solucion = vecina
      mejor_distancia = distancia_vecina
      n = 0
    else:
      n += 1

  print(f"\nNúmero de {iteracion} iteraciones")
  print("Mejor solucion:", solucion)
  print("Distancia     :", mejor_distancia)
  return solucion
```

```
# Generamos una solución aleatoria como punto de partida común
solucion_inicial = crear_solucion(Nodos)
print("Solucion inicial, distancia:", distancia_total(solucion_inicial, problem))

# 1) Búsqueda local simple (solo 2-opt como antes)
print("Busqyeda local (2-opt)")
sol_local = busqueda_local(solucion_inicial, problem)
dist_local = distancia_total(sol_local, problem)

# 2) Búsqueda local con entornos variables (or-opt n=5 hasta n=2 + intercambios)
print("Busqueda local con entornos variables (n-opt, n=5 hasta n=2)")
```

```
sol_vns = busqueda_local_VNS(solucion_inicial, problem)
dist_vns = distancia_total(sol_vns, problem)

# Resultado
print(f"Busqueda Local:     {dist_local}")
print(f"Busqueda Local con entornos variables:        {dist_vns}")
mejora = ((dist_local - dist_vns) / dist_local) * 100
print(f"Ratio de mejora de VNS respecto a BL:            {mejora:.2f}%")
```

```
Solucion inicial, distancia: 4841
Busqyeda local (2-opt)
En la iteracion  30 , la mejor solución encontrada es: [0, 2, 27, 32, 34, 33, 20, 35, 36, 37, 15, 1, 4, 6, 14, 16, 19, 13,
Distancia     : 1813
Busqueda local con entornos variables (n-opt, n=5 hasta n=2)
  Iteracion 1: Mejora con or-opt(5) -> distancia = 4412
  Iteracion 2: Mejora con or-opt(5) -> distancia = 3959
  Iteracion 3: Mejora con or-opt(5) -> distancia = 3701
  Iteracion 4: Mejora con or-opt(5) -> distancia = 3471
  Iteracion 5: Mejora con or-opt(5) -> distancia = 3245
  Iteracion 6: Mejora con or-opt(5) -> distancia = 3038
  Iteracion 7: Mejora con or-opt(5) -> distancia = 2910
  Iteracion 8: Mejora con or-opt(5) -> distancia = 2749
  Iteracion 9: Mejora con or-opt(5) -> distancia = 2609
  Iteracion 10: Mejora con or-opt(5) -> distancia = 2506
  Iteracion 11: Mejora con or-opt(5) -> distancia = 2416
  Iteracion 12: Mejora con or-opt(5) -> distancia = 2327
  Iteracion 13: Mejora con or-opt(5) -> distancia = 2249
  Iteracion 14: Mejora con or-opt(5) -> distancia = 2174
  Iteracion 15: Mejora con or-opt(5) -> distancia = 2099
  Iteracion 16: Mejora con or-opt(5) -> distancia = 2049
  Iteracion 17: Mejora con or-opt(5) -> distancia = 1970
  Iteracion 18: Mejora con or-opt(5) -> distancia = 1926
  Iteracion 19: Mejora con or-opt(5) -> distancia = 1859
  Iteracion 21: Mejora con or-opt(4) -> distancia = 1840
  Iteracion 23: Mejora con or-opt(4) -> distancia = 1812
  Iteracion 25: Mejora con or-opt(4) -> distancia = 1784
  Iteracion 26: Mejora con or-opt(5) -> distancia = 1777
  Iteracion 27: Mejora con or-opt(5) -> distancia = 1724
  Iteracion 28: Mejora con or-opt(5) -> distancia = 1723
  Iteracion 29: Mejora con or-opt(5) -> distancia = 1695
  Iteracion 31: Mejora con or-opt(4) -> distancia = 1678
  Iteracion 33: Mejora con or-opt(4) -> distancia = 1672
  Iteracion 36: Mejora con or-opt(3) -> distancia = 1648
  Iteracion 39: Mejora con or-opt(3) -> distancia = 1627
  Iteracion 42: Mejora con or-opt(3) -> distancia = 1617
  Iteracion 45: Mejora con or-opt(3) -> distancia = 1607
  Iteracion 48: Mejora con or-opt(3) -> distancia = 1600
  Iteracion 51: Mejora con or-opt(3) -> distancia = 1571
  Iteracion 55: Mejora con or-opt(2) -> distancia = 1542
  Iteracion 59: Mejora con or-opt(2) -> distancia = 1516
  Iteracion 63: Mejora con or-opt(2) -> distancia = 1499
  Iteracion 67: Mejora con or-opt(2) -> distancia = 1485
  Iteracion 71: Mejora con or-opt(2) -> distancia = 1469
  Iteracion 75: Mejora con or-opt(2) -> distancia = 1467
  Iteracion 78: Mejora con or-opt(3) -> distancia = 1457
  Iteracion 82: Mejora con or-opt(2) -> distancia = 1455
  Iteracion 87: Mejora con 2-opt -> distancia = 1435
  Iteracion 92: Mejora con 2-opt -> distancia = 1416
  Iteracion 93: Mejora con or-opt(5) -> distancia = 1403
  Iteracion 95: Mejora con or-opt(4) -> distancia = 1378
  Iteracion 100: Mejora con 2-opt -> distancia = 1366
  Iteracion 105: Mejora con 2-opt -> distancia = 1360
  Iteracion 110: Mejora con 2-opt -> distancia = 1356

Número de 115 iteraciones
Mejor solucion: [0, 3, 2, 27, 28, 29, 8, 9, 41, 23, 40, 24, 21, 39, 22, 38, 30, 32, 34, 33, 20, 35, 36, 31, 17, 37, 15, 16
```

Conclusión

Sí, podemos mejorar con operadores de vecindad variables como el descenso de vecindad variable (VND) usando el operador or-opt de n=5 hasta n=2 combinado con el 2-opt original, con este metodo estamos obteniendo mejores resultados que con la búsqueda local simple.

Recocido Simulado

¿Se puede mejorar con otra elección no tan aleatoria (modificando genera_vecina_aleatorio())?

En el recocido simulado original, la función `genera_vecina_aleatorio()` elige dos nodos completamente al azar para intercambiarlos, es aquí donde podemos encontrarnos con el problema de estar moviendo nodos que ya están bien colocados y por lo tanto realizamos iteraciones sin necesidad.

La idea es hacer la selección de cada nodo guiada por el coste local, el cual calculamos como la suma de las distancias a sus dos vecinos en el recorrido. Si detectamos un nodo con coste local alto interpretaremos que está mal colocado, por lo que será más susceptible de ser intercambiado.

```
def genera_vecina_guiada(solucion):
  n = len(solucion)

  # Calculamos coste local de cada nodo (distancia al anterior y al siguiente)
  costes = []
  for i in range(n):
    prev = (i - 1) % n
    sig = (i + 1) % n
    coste = distancia(solucion[prev], solucion[i], problem) + distancia(solucion[i], solucion[sig], problem)
    costes.append(coste)

  # El nodo de inicio no lo movemos, lo dejamos fijo
  costes[0] = 0

  # Elegimos el primer nodo en base al más probable (peor colocado)
  i = random.choices(range(1, n), weights=costes[1:], k=1)[0]

  # El segundo nodo lo elegimos al azar siempre que sea diferente de i
  candidatos = [x for x in range(1, n) if x != i]
  j = random.choice(candidatos)

  if i > j:
    i, j = j, i

  nueva_solucion = solucion.copy()
  nueva_solucion[i], nueva_solucion[j] = nueva_solucion[j], nueva_solucion[i]

  return nueva_solucion
```

```
def recocido_simulado_mejorado(problem, TEMPERATURA):
  sol_actual = crear_solucion(Nodos)
  dist_actual = distancia_total(sol_actual, problem)

  mejor_solucion = []
  mejor_distancia = 10e100

  n = 0
  while TEMPERATURA > .0001:
    n += 1

    # usamos la funcion guiada en vez de la aleatoria
    vecina = genera_vecina_guiada(sol_actual)

    dist_vecina = distancia_total(vecina, problem)

    if dist_vecina < mejor_distancia:
      mejor_solucion = vecina
      mejor_distancia = dist_vecina

    if dist_vecina < dist_actual or probabilidad(TEMPERATURA, abs(dist_actual - dist_vecina)):
      sol_actual = vecina
      dist_actual = dist_vecina

    TEMPERATURA = bajar_temperatura(TEMPERATURA)

  print("La mejor solucion encontrada es " , end="")
  print(mejor_solucion)
  print("con una distancia total de " , end="")
  print(mejor_distancia)

  return mejor_solucion
```

```
temp_inicial = 100000000
n_ejecuciones = 5

print("Recocido Simulado (vecino aleatorio)")
resultados_original = []
for run in range(n_ejecuciones):
  sol_orig = recocido_simulado(problem, temp_inicial)
  resultados_original.append(distancia_total(sol_orig, problem))

print("Recocido Simulado (Vecino guiado)")
resultados_mejorado = []
for run in range(n_ejecuciones):
  sol_mej = recocido_simulado_mejorado(problem, temp_inicial)
  resultados_mejorado.append(distancia_total(sol_mej, problem))
```

```
print("Resultados:")
media_orig = sum(resultados_original) / len(resultados_original)
media_mej = sum(resultados_mejorado) / len(resultados_mejorado)
mejor_orig = min(resultados_original)
mejor_mej = min(resultados_mejorado)

print(f"Original  -> media: {media_orig:.1f}  mejor: {mejor_orig}")
print(f"Mejorado  -> media: {media_mej:.1f}  mejor: {mejor_mej}")
mejora_media = ((media_orig - media_mej) / media_orig) * 100
print(f"Mejora media: {mejora_media:.2f}%")
```

```
Recocido Simulado (vecino aleatorio)
La mejor solución encontrada es [0, 1, 15, 37, 7, 32, 31, 17, 36, 35, 27, 26, 18, 12, 25, 11, 13, 5, 19, 16, 14, 6, 2, 22, 3
con una distancia total de 2098
La mejor solución encontrada es [0, 9, 23, 41, 8, 6, 31, 35, 36, 14, 19, 13, 16, 15, 37, 17, 7, 38, 39, 24, 40, 21, 22, 3, 4
con una distancia total de 2037
La mejor solución encontrada es [0, 10, 25, 41, 23, 40, 24, 22, 38, 33, 34, 28, 30, 29, 39, 21, 9, 8, 14, 37, 15, 16, 11, 12
con una distancia total de 2072
La mejor solución encontrada es [0, 6, 5, 19, 16, 37, 15, 14, 13, 7, 17, 36, 35, 31, 1, 26, 12, 18, 11, 25, 41, 23, 40, 9, 2
con una distancia total de 1923
La mejor solución encontrada es [0, 26, 6, 2, 27, 28, 29, 32, 17, 31, 5, 13, 18, 12, 11, 25, 10, 9, 39, 24, 40, 21, 19, 14,
con una distancia total de 2208
Recocido Simulado (Vecino guiado)
La mejor solucion encontrada es [0, 5, 18, 25, 10, 8, 9, 29, 32, 34, 33, 20, 7, 4, 22, 38, 30, 28, 26, 19, 14, 16, 17, 31, 3
con una distancia total de 2021
La mejor solucion encontrada es [0, 10, 25, 18, 29, 22, 38, 33, 34, 3, 6, 4, 8, 9, 23, 40, 24, 21, 41, 11, 12, 26, 5, 13, 37
con una distancia total de 2166
La mejor solucion encontrada es [0, 29, 39, 21, 24, 40, 9, 8, 11, 12, 5, 26, 13, 16, 19, 14, 15, 1, 4, 3, 27, 28, 23, 41, 10
con una distancia total de 1888
La mejor solucion encontrada es [0, 17, 36, 35, 33, 34, 32, 3, 2, 25, 10, 4, 1, 16, 15, 37, 31, 20, 38, 30, 27, 28, 29, 22,
con una distancia total de 1895
La mejor solucion encontrada es [0, 15, 37, 17, 31, 20, 33, 34, 19, 14, 16, 13, 11, 12, 25, 41, 40, 23, 10, 9, 21, 24, 39, 2
con una distancia total de 1958
Resultados:
Original  -> media: 2067.6  mejor: 1923
Mejorado  -> media: 1985.6  mejor: 1888
Mejora media: 3.97%
```

Conclusión

Sí, se puede mejorar el recocido simulado modificando `genera_vecina_aleatorio()`. Usando una selección guiada por el coste local evitamos que los nodos que están peor colocados tengan menos probabilidad de ser seleccionados para el intercambio. Conseguimos que los intercambios sean mejores y no se pierdan tantas iteraciones moviendo nodos que ya estaban en buena posición.